

Construção eficiente de kd-trees para primitivas triangulares

Alessandro Ribeiro da Silva¹, Wallace Santos Lages¹

¹UFMG - Universidade Federal de Minas Gerais
asilva@dcc.ufmg.br, wsl@dcc.ufmg.br

Resumo

Atualmente as árvores binárias multidimensionais (kd-trees) são a escolha de muitas aplicações onde é necessário realizar pesquisas eficientes pelos k vizinhos mais próximos ou testes de interseção com raios. Entretanto, a maioria dos algoritmos para construção de árvores otimizadas para o tempo de pesquisa é da ordem $O(N \log^2 N)$ ou $O(N^2)$. Neste artigo procuramos sumarizar o estado da arte na construção de kd-trees para primitivas triangulares apresentando um algoritmo para o particionamento em $O(N \log N)$.

1. Introdução

Problemas que envolvem determinação de vizinhança e testes de interseção estão presentes em diversas áreas como GIS (Geographic Information System), photon mapping [9], planejamento de trajetórias de robôs[3], raytracing [7], jogos gráficos [1].

Uma solução eficiente para estes problemas é armazenar os dados espacialmente, de forma a minimizar o tempo de pesquisa. Algumas estruturas de particionamento espacial conhecidas são: grids, BSP (binary space partitioning), quad-trees, octrees, kd-trees, etc.

O Grid uniforme é uma divisão regular do espaço S delimitado por um voxel V em subvoxels menores e de mesma dimensão [5]. Apesar de ser simples para construir e percorrer, os grids geram voxels mesmo onde não existem dados, tornando a pesquisa mais custosa e a utilização de memória pouco eficiente

BSP é uma árvore binária que divide o espaço S recursivamente em 2 sub-espacos por meio de um plano. Ao contrário da grid, uma BSP permite divisões não uniformes, o que possibilita uma melhor adaptação à distribuição espacial dos dados. O único inconveniente desta estrutura é manipulação de planos com orientações arbitrárias, o que pode transformar a decisão de posicionamento dos planos de divisão uma tarefa difícil [5].

Quadtree é uma árvore que divide o plano em quatro subplanos. As linhas de divisão são alinhadas aos eixos de coordenadas e são aplicadas recursivamente como nas BSPs. As Octrees são o equivalente tridimensional das quadtrees, dividindo o volume em oito sub-volumes por meio de quatro planos [5].

Kd-tree é uma variante da árvore BSP na qual os planos que dividem os sub-espacos devem estar alinhados aos eixos do sistema de coordenadas utilizado [4]. Ao contrário das quadtree e octree a kd-tree pode ser aplicada a qualquer dimensão. As kd-trees estão sem dúvida entre os métodos mais eficientes conhecidos para pesquisas pelos k vizinhos mais

próximos ou testes de interseção com raios. Elas são utilizadas extensivamente em aplicações de photon mapping, CAD, raytracing etc. Frequentemente as primitivas dessas aplicações são representadas por triângulos.

Na maioria dos trabalhos apresentados sobre kd-trees em primitivas triangulares o enfoque dado está na otimização de seu tempo de pesquisa. Wald e Havran [8] apresentam uma abordagem com foco na construção da mesma, visando um algoritmo para construção em tempo $O(N \log N)$, que é o limite inferior para construção de kd-trees.

Como na BSP, a velocidade da árvore nas pesquisas está relacionada com uma boa escolha dos planos de particionamento. A técnica simplificada tende a construir árvores balanceadas, mas nem sempre a melhor árvore deve possuir esta característica. No caso do raytracing [8] e photon mapping a construção de árvores não balanceadas proporcionam um melhor desempenho para estas técnicas. Infelizmente, a construção de árvores de boa qualidade não é trivial.

Antes de apresentar o algoritmo ótimo para construção, vamos apresentar um método simples para a construção de kd-trees, mas que não gera uma árvore de boa qualidade. Em seguida apresentaremos a heurística mais utilizada para a construção de árvores kd-trees e como utilizá-la para construir árvores eficientes em tempo mínimo.

2 kd-tree

Kd-tree é uma árvore binária k -dimensional com os planos de divisão alinhados aos eixos do sistema de coordenada. É utilizada para indexação espacial permitindo operações como range search, determinação de vizinhança, interseções com triângulos [4], etc.

Supondo um espaço S contendo T triângulos, uma kd-tree sobre este espaço é uma árvore binária dada pela divisão recursiva de S . O nó raiz corresponde a um AABB (Axis Aligned BoundBox - caixa envolvente alinhada aos eixos) sobre S .

Os nós internos da árvore correspondem aos planos p que dividem S em uma determinada dimensão k sobre uma posição e , formalmente, $p_{k,e}(x) : x_k = e$ [8].

2.1 Construindo uma kd-tree

Para muitas aplicações atuais (ex: raytracing) o tempo de construção das kd-trees começa a se tornar um fator importante, uma vez que os dados utilizados vem se tornando maiores e mais complexos (grande quantidade de triângulos). Como exemplo, pode-se citar a renderização *sunflower*[1] que possui um bilhão de triângulos (Figura 1).



Figura 1. Renderização de uma cena de aproximadamente 1 bilhão de triângulos por um ray-tracer.

O algoritmo 1 é uma versão recursiva para construção de uma kd-tree.

Algorithm 1 Algoritmo recursivo para construção de kd-tree.

```

function RECBUILD(Primitive T, voxel V)
  if Terminate(T,V) then
    return leaf(T)
  end if
   $p = \text{FindPlane}(T, V)$  procura melhor divisão
   $(V_L, V_R) = \text{divida } V \text{ em } p$ 
   $T_L = \{t \in T | (t \cap V_L) \neq \emptyset\}$ 
   $T_R = \{t \in T | (t \cap V_R) \neq \emptyset\}$ 
  return node(p, recBuild( $T_L, V_L$ ), recBuild( $T_R, V_R$ ))
end function

function BUILDKDTREE(primitives[] T)
   $V = \text{AABB}(T)$ 
  return recBuild(T,V)
end function

```

Existem duas situações, que devem ser observados na construção, que influenciam diretamente o desempenho final da árvore:

- **Avaliação de um critério de parada da recursão:** este critério é trivial de ser especificado, pois podemos simplesmente parar o crescimento da árvore depois de n recursões. Uma variável para se tomar esta decisão pode ser a quantidade máxima de memória que a árvore pode alocar.
- **Seleção da posição do melhor plano de partição:** esta seleção pode ser simples, mas geralmente gera árvores não eficientes. Pode requerer uma análise aprofundada sobre as possibilidades de divisões espaciais, o que torna sua implementação inviável. Atualmente tem-se utilizado a heurística SAH [5] para determinação do posicionamento do plano de partição.

Para os testes de interseções de raios, uma divisão arbitrária pode criar voxels desnecessários, gerando testes excessivos enquanto se percorre a árvore [8].

3 Particionamento de kd-tree

Neste artigo é dada uma atenção maior para o processo de partição pois o mesmo, como visto anteriormente, não é trivial para a construção eficiente de uma kd-tree otimizada.

De acordo com Wald e Havran [8] existem diversas heurísticas para se construir kd-trees, mas cada uma delas possui peculiaridades que as tornam muito dependentes de parâmetros da cena. Por este motivo uma nova abordagem baseada na heurística de área da superfície (SAH) foi apresentada e dentre todas as outras heurísticas já propostas até então, foi a que obteve um melhor resultado.

A seguir é descrita uma técnica simples de divisão que gera árvores não otimizadas, logo após é apresentado a técnica SAH que possui uma abordagem mais elaborada.

3.1 Dividir pela mediana

A divisão pela mediana é uma técnica amplamente utilizada para construção de kd-trees [8] principalmente por sua facilidade de implementação.

Esta divisão consiste em escolher o plano que está posicionado no meio do voxel em relação a uma determinada dimensão dim . Na próxima aplicação da divisão a dimensão dim de referência é incrementada de acordo com a relação:

$$dim = (dim + 1) \bmod k$$

A posição do plano de divisão p_e sobre a dimensão dim dentro de um voxel V é determinada de acordo com a relação:

$$p_e = \frac{1}{2}(V_{min,dim} + V_{max,dim})$$

A recursão acaba quando o número de elementos por voxel chega a uma quantidade máxima especificada $triMax$ ou quando o número de recursões D atinge um limite $maxDepth$.

$$Terminate(T, V) = (|T| \leq triMax) \vee (D \geq maxDepth)$$

3.2 Heurística da área superficial (SAH)

A técnica SAH é baseada no cálculo de uma função de custo que considera a estrutura geométrica do voxel V ao ser dividido por um plano p estimando o custo de percorrimento na árvore por um determinado raio no espaço.

Esta heurística assume três condições (não necessariamente verdadeiras):

- Os raios estão distribuídos uniformemente e estes são considerados como retas que não começam, terminam e nem são bloqueadas dentro de um voxel.
- O custo de percorrimento (atravessamento) da árvore e o custo de interseção com um elemento dentro do nó são conhecidos. Sendo K_T , e K_I respectivamente.
- O custo de interseção com N elementos é aproximadamente NK_I .

De acordo com Wald e Havran [8] a probabilidade P de um raio de um raio que intercepta um voxel V interceptar também um sub-voxel é:

$$P_{[V_{sub}|V]} = \frac{SA(V_{sub})}{SA(V)} \quad (1)$$

onde $SA(X)$ é a área da superfície do voxel X .

O custo esperado para uma divisão realizada através de um plano p sobre um voxel V é dado pelo custo de um atravessamento K_T somado ao custo de atravessar os dois filhos multiplicado pelas respectivas probabilidades:

$$C(p) = K_T + P_{[V_L|V]}C(V_L) + P_{[V_R|V]}C(V_R) \quad (2)$$

3.3 SAH gulosa

Utilizando a SAH é possível estimar o custo de uma cena completa. Aplicando a equação recursivamente a partir da raiz obtemos:

$$C(\tau) = \sum_{n \in \text{nos}} \frac{SA(V_n)}{SA(V_S)} K_t + \sum_{l \in \text{folhas}} \frac{SA(V_l)}{SA(V_S)} K_i \quad (3)$$

Onde V_s é AABB da cena S . Entretanto, minimizar a equação 3 não é trivial porque o espaço de árvores possíveis cresce muito rapidamente com o tamanho da cena. Desse modo, ao invés da solução ótima, considera-se que todos os filhos resultantes são folhas:

$$C(p) = K_t + K_i \left(\frac{SA(V_l)}{SA(V)} |T_l| + \frac{SA(V_r)}{SA(V)} |T_r| \right) \quad (4)$$

Apesar de ser uma simplificação grande, nenhuma outra aproximação com resultado sistematicamente melhor foi encontrada até agora [8].

4 Construindo kd-trees para primitivas triangulares

Considerando que uma subdivisão de N triângulos gera duas listas de tamanho $N/2$ e que a recursão para em $N = 1$ podemos ver que o custo mínimo teórico para se construir uma kd-tree é $O(N \log N)$. Entretanto, a construção de uma kd-tree utilizando uma SAH é bem mais complexa e pode levar facilmente a soluções de complexidade alta.

4.1 Algoritmo de particionamento $O(N^2)$

É possível perceber que a função custo é contínua exceto nos pontos onde um triângulo entra ou sai do plano de divisão. Podemos então considerar apenas os planos que passam por esses pontos. Eles recebem o nome de *planos candidatos à divisão*. Para cada um deles é preciso determinar o número de triângulos no lado esquerdo N_l , direito N_r e no plano N_p para que se possa calcular o custo por meio da SAH.

Infelizmente, é preciso classificar todos os triângulos para cada plano candidato. Como podem haver até 6N planos candidatos por triângulo o custo total para se achar o melhor plano no algoritmo é $O(N^2)$. Esse custo persiste durante a recursão, fazendo que o custo total de construção continue pertencente a $O(N^2)$.

4.2 Algoritmo de particionamento $N \log^2 N$

Claramente o alto custo de particionamento do algoritmo anterior se deve ao custo de se computar os valores de N_L, N_R

Algorithm 2 Algoritmo de particionamento $O(N^2)$

```

function PARTITION(triangles $T$ , voxel $V$ )
  for all  $t \in T$  do
     $(\hat{C}, \hat{p}_{side}) = (\infty, 0)$ 
    for all  $p \in \text{SplitCandidates}$  do
       $(V_L, V_R) = \text{divida } V \text{ em } p$ 
       $(T_L, T_R, T_p) = \text{Classify}(T, V_L, V_R, p)$ 
       $(C, p_{side}) = \text{SAH}(V, p, |T_L|, |T_R|, |T_p|)$ 
      if  $(C < \hat{C})$  then
         $(\hat{C}, \hat{p}_{side}) = (C, p_{side})$ 
      end if
       $(T_l, T_r, T_p) = \text{Classify}(T, V_l, V_r, p)$ 
      if  $(p_{side} = \text{LEFT})$  then
        return  $(\hat{p}, T_l \cup T_p, T_r)$ 
      else
        return  $(\hat{p}, T_l \cup T_r, T_p)$ 
      end if
    end for
  end for
end function

function BUILDKDTREE(triangles $T$ )
   $V = \text{AABB}(T)$ 
  return  $\text{recBuild}(T, V)$ 
end function

```

e N_P . Se os triângulos estivessem ordenados seria possível calcular esses valores incrementalmente à medida que avançamos o plano de divisão. Utilizando essa idéia é possível construir uma variação do Algoritmo 2 capaz de construir a mesma árvore em $O(N \log^2 N)$.

Para facilitar a classificação dos triângulos, a medida que cada plano candidato é considerado, armazenamos em uma lista E os eventos que seriam gerados para os triângulos daquele ponto. Se um triângulo está perpendicular ao eixo sendo considerado, ele gera um evento *planar* $(t, B_{k,min}, |)$. Caso contrário ele gera um evento de *início* $(t, B_{k,min}, +)$ e um evento de *fim* $(t, B_{k,max}, -)$. Cada evento $e = (e_t, e_\xi, e_{type})$ consiste de uma referência ao triângulo que o gerou, a posição e_ξ do plano, uma flag e_{type} especificando se t começa, termina ou é planar ao plano.

Uma vez que tenham sido gerados os eventos para todos os triângulos, a lista E é ordenada de maneira crescente de acordo com a posição do plano e tal que para os planos na mesma posição, os eventos de *fim* precedam os eventos de *planar* que por sua vez precedam os eventos de *início*. Dessa forma, se contarmos o número de eventos de cada plano i , teremos p_i^+ , $p_i^|$ e p_i^- que denotam respectivamente o número de triângulos começando, terminando ou no plano em questão. A partir daí, os valores de N_L , N_R e N_P podem ser derivados com um regra simples e o valor do custo do plano calculado imediatamente:

$$N_L^{(i)} = N_L^{(i-1)} + p_{i-1}^| + p_{i-1}^+$$

$$N_R^{(i)} = N_R^{(i-1)} + p_i^| + p_i^-$$

$$N_P^{(i)} = p_{i-1}^|$$

Neste algoritmo são feitas $O(N)$ chamadas a SAH e para calcular os valores P^+ , P^- e $P^|$ no loop interno. Além disso, para cada triângulo é feita uma ordenação com custo estimado

Algorithm 3 Algoritmo incremental para se encontrar $\hat{p}$

```
function FINDPLANE(triangles $T$ , voxel $V$ )
   $(\hat{C}, \hat{p}) = (\infty, 0)$ 
  Considera cada uma das dimensões
  for  $k = 1..3$  do
    eventlist $E = 0$ 
    for all  $t \in T$  do
       $B = \text{ClipTriangleToBox}(t, V)$ 
      if  $B$  is Planar then
         $E = E \cup (t, B_{\min, k}, |)$ 
      else
         $E = E \cup (t, B_{\min, k}, +) \cup (t, B_{\max, k}, -)$ 
      end if
      sort( $E, < E$ ) Ordena todos os planos
    end for
  end for

  Varre o plano em todos os planos candidatos
   $N_l = 0, N_p = 0, N_r = |T|$ 
  for  $i = 0; i < |E|$  do
     $p = E_{i,p}, p^+ = P^- = p^- = 0$ 
    while  $(i < |W|)$  do
      while  $(E_{i,\xi} = p_\xi) \cap (E_{i,type} = -)$  do
         $P^- += 1$ 
      end while
      while  $(E_{i,\xi} = p_\xi) \cap (E_{i,type} = |)$  do
         $P^+ += 1$ 
      end while
      while  $(E_{i,\xi} = p_\xi) \cap (E_{i,type} = +)$  do
         $P^+ += 1$ 
      end while
       $i += 1$ 
    end while
     $N_P = P^+, N_{R-} = P^-, N_{R+} = P^-$ 
     $(C, p_{side}) = \text{SAH}(V, p, N_L, N_R, N_P)$ 
    if  $(C < \hat{p}_c)$  then
       $(\hat{C}, \hat{p}, \hat{p}_{side}) = (C, p, p_{side})$ 
       $N_{L+} = P^+, N_{L-} = P^-, N_P = 0$ 
    end if
  end for
  return  $(\hat{p}, \hat{p}_{side})$ 
end function
```

$O(N \log N)$ ¹. Assim a complexidade é dominada pela ordenação. Quando aplicada recursivamente esse algoritmo leva a uma complexidade da ordem de $O(n \log^2 n)$

4.3 Algoritmo de particionamento $O(N \log N)$

O algoritmo de particionamento $O(N \log^2 N)$ já é um pouco melhor, mas ainda longe do mínimo $O(N \log N)$. A razão é que a lista de triângulos deve ser reordenada a cada iteração. Se fosse possível manter a lista ordenada após uma primeira ordenação de modo que não fosse preciso reordená-la para cada filho, o custo recursivo poderia cair até o ótimo.

Wald e Habran propõem em [8] uma maneira de construir as listas E_L e E_R para os filhos sem reordenação. Eles se basearam nos seguintes fatos:

- É possível iterar várias vezes sobre T e E e continuar com o custo em $O(N)$ se o número de iterações for constante.
- Classificar todos os triângulos em T_L ou T_R pode ser feito em $O(N)$
- Uma vez que E esteja ordenada, qualquer sub lista estará ordenada também.
- Duas listas ordenadas de tamanho $O(N)$ podem ser unidas em uma terceira lista ordenada em uma iteração mergesort.
- Os triângulos que estão completamente em um lado do plano terão os mesmos eventos que o nó atual.
- Os triângulos que cruzarem o plano geram eventos para ambos os lados.
- Para cenas razoáveis, existem no máximo $O(\sqrt{N})$ triângulos cruzando o plano [2].

Primeiro os eventos são gerados para todos os triângulos em todas as dimensões. Como todos os eventos serão tratados no mesmo laço, é necessário armazenar uma cópia de N_L , N_R e N_P para cada dimensão. Em seguida os eventos são ordenados pela posição p_ξ do evento. Os eventos com a mesma posição são ordenados de maneira que os eventos da mesma dimensão k estejam consecutivos. Finalmente, os eventos no mesmo plano e com a mesma dimensão são ordenados de modo que os eventos *fim* venham primeiro, seguidos dos eventos *planar* e dos eventos *início*.

A partir dessa lista previamente ordenada de eventos, é possível realizar a classificação e construção da lista eventos dos filhos E_L e E_R sem reordenar.

- **Classificação** Uma vez que o plano tenha sido encontrado é preciso determinar para cada triângulo, se ele pertence a T_L, T_R ou ambos. Primeiro todos os triângulos são marcados como pertencentes a ambos os lados. Para que um triângulo pertença apenas a T_L , ele deve: estar à esquerda do plano ou interceptar o plano pela esquerda; ser paralelo ao plano e estar à esquerda do mesmo ou estar no conjunto T_P com o $\hat{p}_{side} == LEFT$. Para o lado direito, critérios similares são aplicáveis. Se o triângulo preencher algum desses requisitos ele é marcado como pertencente a apenas um lado.

¹Teoricamente é possível utilizar um algoritmo *bucketsort*, com complexidade assintótica linear

- **Divisão de E em E_{LO} e E_{RO}** Triângulos que não interceptam $\hat{p}$ geram eventos para um lado. Desse modo, uma vez que E já tenha sido classificada, é possível percorrer E novamente colocando todos os eventos correspondentes a um triângulo totalmente à esquerda na lista E_{LO} e todos os triângulos totalmente à direita na lista E_{RO} . Eventos que afetam ambos os lados são naturalmente descartados e as novas listas criadas são criadas já ordenadas.
- **Geração dos eventos para os triângulos que interceptam o plano.** Os triângulos que interceptam o plano geram eventos para ambos os lados. Esses triângulos são então comparados com o voxel e os novos eventos são inseridos nas listas E_{BL} e E_{BR} . Esses eventos são gerados em uma ordem não controlada.
- **União das quatro listas** Para produzir as listas finais E_L e E_R as quatro listas criadas devem ser fundidas. Para isso primeiramente as listas E_{BL} e E_{BR} devem ser ordenadas. Assumindo que apenas $O(\sqrt{N})$ triângulos interceptam $\hat{p}$ o custo de ordenar essas listas vai ser $O(|E_{LO}|\log|E_{LO}|) = O(\sqrt{N}\log\sqrt{N}) \subset O(\sqrt{N}x\sqrt{N}) = O(N)$. Uma vez que elas estejam ordenadas, basta um passo mergesort para uni-las em $O(N)$.

Algorithm 4 Algoritmo de varredura para classificar triângulos em Direita, Esquerda ou Plano.

```

function CLASSIFY2(triangles $T$ , eventlist $E$ , plane $\hat{p}$ )
  for all  $t \in T$  do
     $t_{side} = Both$ 
    for all  $e \in E$  do
      if ( $e_{type} = - \wedge e_k = \hat{p}_k \wedge e_\xi \leq \hat{p}_\xi$ ) then
         $t[e_t]_{side} = LeftOnly$ 
      else if ( $e_{type} = + \wedge e_k = \hat{p}_k \wedge e_\xi \geq \hat{p}_\xi$ ) then
         $t[e_t]_{side} = RightOnly$ 
      else if ( $e_{type} = | \wedge e_k = \hat{p}_k$ ) then
        if ( $e_\xi < \hat{p}_\xi$ ) then
           $t[e_t]_{side} = LeftOnly$ 
        end if
        if ( $e_\xi = \hat{p}_\xi \wedge \hat{p}_{side} = LFT$ ) then
           $t[e_t]_{side} = LeftOnly$ 
        end if
        if ( $e_\xi > \hat{p}_\xi$ ) then
           $t[e_t]_{side} = RightOnly$ 
        end if
        if ( $e_\xi = \hat{p}_\xi \wedge \hat{p}_{side} = RIGHT$ ) then
           $t[e_t]_{side} = RightOnly$ 
        end if
      end if
    end for
  end for
end function

```

Observe que ainda é necessário realizar a ordenação dos eventos, mas ela é realizada apenas uma vez. A partir daí todos os passos são realizados em $O(N)$. Quando aplicada recursivamente, obtemos $O(N\log N)$ que é o limite mínimo teórico.

5 Conclusão e Trabalhos futuros

Mostramos que apesar de não ser trivial, é possível construir kd-trees eficiente para dados triangulares utilizando a heurística de área superficial (SAH). Apesar de ser mais eficiente, Wald reporta que o tempo de relógio entre as versões $O(N\log^2 N)$ e $O(N\log N)$ diferem de apenas uma ordem de grandeza mesmo para cenas com 4 milhões de triângulos [8].

Isso provavelmente implica que as constantes associadas a construção das listas de eventos e avaliação dos custos dos planos são bem maiores que as associadas a ordenação da lista. Uma contribuição interessante envolveria estimar o valor das constantes empiricamente e propor métodos para acelerar as outras etapas.

É importante notar também que a construção mais rápida atual ainda demora segundos ou minutos para cenas de complexidade razoável [6]. Isso torna inviável a reconstrução iterativa das árvores para cenários dinâmicos, fazendo com que kd-trees sejam restritas a cenários estáticos. Como a complexidade de construção já atingiu o ótimo (a menos de constantes), alguns pesquisadores tem propostos modificações para permitir animações restritas dentro da kd-tree. Uma abordagem alternativa para o problema seria avaliar a possibilidade de realizar reconstruções parciais na kd-tree, modificando a hierarquia ao longo das pesquisas.

Referências

- [1] Realtime raytracing project
url: “<http://www.openrt.de/>”. acessado em: 16 de junho de 2006.
- [2] Van der Stappen de Berg M. T, Katz M J. Realistic input models for geometric algorithms. *Algorithmica* 34, 2002.
- [3] James Bruce e Manuela Veloso. Real-time randomized path planning for robot navigation. volume 30, pages 170–231, 2002.
- [4] Volker Gaede and Oliver Günther. Multidimensional access methods. *ACM Computing Surveys*, 30(2):170–231, 1998.
- [5] Vlastimil Havran. *Heuristic Ray Shooting Algorithms*. PhD thesis, Faculty of Electrical Engineering, Czech Technical University, 2000. Available at <http://www.cgg.cvut.cz/havran/DISSVH/dissvh.pdf>.
- [6] Ingo Wald Hans-Peter Seide e Phillip Slusallek Johannes Gunter, Heiko Friedrich. Ray tracing animated scenes using motion decomposition. *Eurographics 2006, Vol 25, Number 3*, 2006.
- [7] Ingo Wald. *Realtime Ray Tracing and Interactive Global Illumination*. PhD thesis, Computer Graphics Group, Saarland University, 2004. Available at <http://www.mpi-sb.mpg.de/~wald/PhD/>.
- [8] Ingo Wald. On building fast kd-trees for ray tracing, and on doing that in $O(N \log N)$. *Technical Report, SCI Institute, University of Utah, No UUSCI-2006-009 (submitted for publication)*, 2006.

- [9] Ingo Wald, Johannes Günther, and Philipp Slusallek. Balancing Considered Harmful – Faster Photon Mapping using the Voxel Volume Heurist. *Computer Graphics Forum*, 22(3):595–603, 2004. (Proceedings of Eurographics).